

1.事务底层与高可用原理

事务的基础知识

mysql的事务分为显式事务和隐式事务

- 默认的事务是隐式事务
- 显式事务由我们自己控制事务的开启，提交，回滚等操作

```
show variables like 'autocommit';
```

```
mysql> show variables like 'autocommit';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| autocommit    | ON    |
+-----+-----+
1 row in set (0.00 sec)
```

事务基本语法

事务开始

- 1、begin
- 2、START TRANSACTION (推荐)
- 3、begin work

事务回滚

rollback

事务提交

commit

使用事务插入两行数据，commit后数据还在

```

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into testdemo values(5,5,5);
Query OK, 1 row affected (0.00 sec)

mysql> insert into testdemo values(6,6,6);
Query OK, 1 row affected (0.00 sec)

mysql> select * from testdemo
-> ;
+-----+-----+-----+
| id | order_id | goods_id |
+-----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+-----+-----+-----+
| id | order_id | goods_id |
+-----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+-----+-----+-----+
2 rows in set (0.00 sec)

```

使用事务插入两行数据，rollback后数据没有了

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into testdemo values(7,7,7);
Query OK, 1 row affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from testdemo;
+-----+-----+-----+
| id | order_id | goods_id |
+-----+-----+-----+
| 5 | 5 | 5 |
| 6 | 6 | 6 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

1.1.redo日志

在事务的实现机制上，MySQL采用的是WAL（Write-ahead logging，预写式日志）机制来实现的。

就是所有的修改都先被写入到日志中，然后再被应用到系统中。通常包含redo和undo两部分信息。

redo log称为重做日志，每当有操作时，在数据变更之前将操作写入redo log，这样当发生掉电之类的情況时系统可以在重启后继续操作。

undo log称为撤销日志，当一些变更执行到一半无法完成时，可以根据撤销日志恢复到变更之间的状态。

MySQL中用redo log来在系统Crash重启之类的情況时修复数据（事务的持久性），而undo log来保证事务的原子性。

1.1.1.redo日志及作用

1.1.1.1.redo日志

MySQL的数据目录（使用SHOW VARIABLES LIKE 'datadir'查看）下默认有两个名为ib_logfile0和ib_logfile1的文件，这个就是redo日志

```
mysql> show variables like 'datadir';
+-----+-----+
| Variable_name | Value          |
+-----+-----+
| datadir       | /var/lib/mysql/ |
+-----+-----+
1 row in set (0.00 sec)
```

```
[root@centosvm mysql]# ls -all
总用量 123040
drwxr-x--x.  6 mysql mysql    4096 6月  27 13:26 .
drwxr-xr-x. 59 root  root    4096 6月  27 13:25 ..
-rw-r-----. 1 mysql mysql     56 5月  13 18:17 auto.cnf
-rw-----. 1 mysql mysql    1676 5月  13 18:17 ca-key.pem
-rw-r-----. 1 mysql mysql    1112 5月  13 18:17 ca.pem
-rw-r-----. 1 mysql mysql     796 6月  14 21:43 centosvm.log
-rw-r-----. 1 mysql mysql   19217 6月  14 21:47 centosvm-slow.log
-rw-r-----. 1 mysql mysql    1112 5月  13 18:17 client-cert.pem
-rw-----. 1 mysql mysql    1676 5月  13 18:17 client-key.pem
-rw-r-----. 1 mysql mysql     423 6月  17 19:24 ib_buffer_pool
-rw-r-----. 1 mysql mysql 12582912 6月  27 13:26 ibdata1
-rw-r-----. 1 mysql mysql 50331648 6月  27 13:26 ib_logfile0
-rw-r-----. 1 mysql mysql 50331648 5月  13 18:17 ib_logfile1
-rw-r-----. 1 mysql mysql 12582912 6月  27 13:26 ibtmp1
drwxr-x---. 2 mysql mysql     193 6月  14 21:15 lijin
drwxr-x---. 2 mysql mysql    4096 6月  10 18:38 mysql
-rw-r-----. 1 mysql mysql    1380 6月  13 10:21 mysql-bin.000001
```

可以通过下边几个启动参数来调节：

`innodb_log_group_home_dir`，该参数指定了redo日志文件所在的目录，默认值就是当前的数据目录。

```
mysql> show variables like 'innodb_log_group_home_dir';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_group_home_dir | ./ |
+-----+-----+
1 row in set (0.01 sec)
```

`innodb_log_file_size`，该参数指定了每个redo日志文件的大小，默认值为48MB，

```
mysql> show variables like 'innodb_log_file_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_file_size | 50331648 |
+-----+-----+
1 row in set (0.00 sec)
```

`innodb_log_files_in_group`，该参数指定redo日志文件的个数，默认值为2，最大值为100。

```
mysql> show variables like 'innodb_log_files_in_group';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_files_in_group | 2 |
+-----+-----+
1 row in set (0.00 sec)
```

所以磁盘上的redo日志文件可以不只一个，而是以一个日志文件组的形式出现的。这些文件以 `ib_logfile[数字]`（数字可以是0、1、2...）的形式进行命名。在将redo日志写入日志文件组时，是从 `ib_logfile0` 开始写，如果 `ib_logfile0` 写满了，就接着 `ib_logfile1` 写，同理，`ib_logfile1` 写满了就去写 `ib_logfile2`，依此类推。如果写到最后一个文件也满了咋办？那就重新转到 `ib_logfile0` 继续写（覆盖

写)。

1.1.1.2.redo日志的作用

在Buffer Pool的时候说过，在真正访问MySQL数据之前，需要把在磁盘上的页缓存到内存中的Buffer Pool之后才可以访问。如果我们只在内存的Buffer Pool中修改了页面，假设在事务提交后突然发生了某个故障，导致内存中的数据都失效了，那么这个已经提交了的事务对数据库中所做的更改也就跟着丢失了，这是我们所不能忍受的。那么如何保证这个持久性呢？一个很简单的做法就是在事务提交完成之前把该事务所修改的所有页面都刷新到磁盘，但是这个做法有以下问题：

- **刷新一个完整的数据页太浪费了**

有时候我们仅仅修改了某个页面中的一个字节，但是我们知道在InnoDB中是以页为单位来进行磁盘IO的，也就是说我们在该事务提交时不得不将一个完整的页面从内存中刷新到磁盘，我们又知道一个页面默认是16KB大小，只修改一个字节就要刷新16KB的数据到磁盘上显然是太浪费了。

- **随机IO刷起来比较慢**

一个事务可能包含很多语句，即使是一条语句也可能修改许多页面，该事务修改的这些页面可能并不相邻，这就意味着在将某个事务修改的Buffer Pool中的页面刷新到磁盘时，需要进行很多的随机IO，随机IO比顺序IO要慢，尤其对于传统的机械硬盘来说。

怎么办呢？我们只是想让已经提交了的事务对数据库中数据所做的修改永久生效，即使后来系统崩溃，在重启后也能把这种修改恢复出来。所以我们其实没有必要在每次事务提交时就把该事务在内存中修改过的全部页面刷新到磁盘，只需要把修改了哪些东西记录一下就好。

比方说某个事务将系统表空间中的第100号页面中偏移量为1000处的那个字节的值1改成2我们只需要记录一下：

将第0号表空间的100号页面的偏移量为1000处的值更新为2。

以上内容也被称之为重做日志，英文名为redo log，也可以称之为redo日志。与在事务提交时将所有修改过的内存中的页面刷新到磁盘相比，只将该事务执行过程中产生的redo日志刷新到磁盘的好处如下：

1、redo日志占用的空间非常小

存储表空间ID、页号、偏移量以及需要更新的值所需的存储空间是很小的。

2、redo日志是顺序写入磁盘的

在执行事务的过程中，每执行一条语句，就可能产生若干条redo日志，这些日志是按照产生的顺序写入磁盘的，也就是使用顺序IO。

1.1.2.redo日志格式

通过上边的内容我们知道，redo日志本质上只是记录了一下事务对数据库做了哪些修改。InnoDB们针对事务对数据库的不同修改场景定义了多种类型的redo日志，但是绝大部分类型的redo日志都有下边这种通用的结构：

type	space ID	page Number	data
------	----------	-------------	------

各个部分的详细释义如下：

type：该条redo日志的类型，redo日志设计大约有53种不同的类型日志。

space ID：表空间ID。

page number: 页号。

data: 该条redo日志的具体内容。

1.1.2.1.简单的redo日志类型

如果某张表没有主键，并且没有定义不允许存储NULL值的UNIQUE键，那么InnoDB会自动为表添加一个名为row_id的隐藏列作为主键。

为这个row_id隐藏列进行赋值的方式如下：

- 内存中维护一个全局变量，当向某个包含row_id隐藏列的表中插入一条记录时，就会把这个全局变量的值当做新记录的row_id的值，并且把这个全局变量+1；
- 每当这个全局变量的值为256的倍数时，就会将该变量的值刷新到系统表空间页号为7的页面中一个名为Max Row Id的属性中。此时需要把这次对这个页面的修改以redo日志的形式记录下来
- 当系统启动时，会将这个Max Row Id属性加载到内存中。

InnoDB把这种极其简单的redo日志称之为物理日志，并且根据在页面中写入数据的多少划分了几种不同的redo日志类型：

MLOG_1BYTE (type=1)

表示在页面的某个偏移量处写入1字节的redo日志类型。

MLOG_2BYTE (type=2)

表示在页面的某个偏移量处写入2字节的redo日志类型。

MLOG_4BYTE (type=4)

表示在页面的某个偏移量处写入4字节的redo日志类型。

MLOG_8BYTE (type=8)

表示在**页面的某个偏移量**处写入**8字节**的redo日志类型

我们上边提到的Max Row ID属性实际占用8个字节的存储空间，所以在修改页面中的该属性时，会记录一条类型为MLOG_8BYTE的redo日志，MLOG_8BYTE的redo日志结构如下所示：

type	space ID	page Number	offset	data
------	----------	-------------	--------	------

offset代表在**页面中的偏移量**。

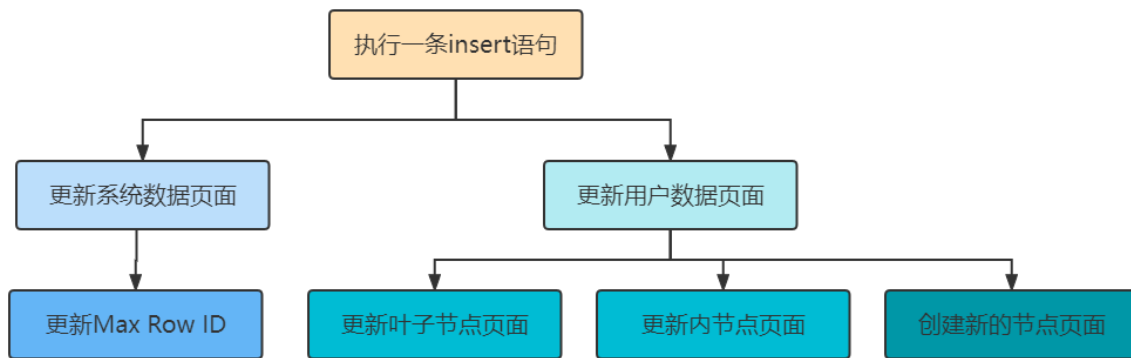
1.1.2.2.复杂的redo日志类型

有时候执行一条语句会修改非常多的页面，包括系统数据页面和用户数据页面（用户数据指的就是聚簇索引和二级索引对应的B+树）。以一条INSERT语句为例，它除了要向B+树的页面中插入数据，也可能更新系统数据Max Row ID的值，不过对于我们用户来说，平时更关心的是语句对B+树所做更新：

表中包含多少个索引，一条INSERT语句就可能更新多少棵B+树。

针对某一棵B+树来说，既可能更新叶子节点页面，也可能更新非叶子节点页面，也可能创建新的页面（在该记录插入的叶子节点的剩余空间比较少，不足以存放该记录时，会进行页面的分裂，在非叶子节点页面中添加目录项记录）。

画一个复杂的redo日志的示意图就像是这样：



大家只要记住：redo日志会把事务在执行过程中对数据库所做的所有修改都记录下来，在之后系统崩溃重启后可以把事务所做的任何修改都恢复出来。

1.1.3.redo日志的写入过程

1.1.3.1.redo log block和日志缓冲区

InnoDB为了更好的进行系统崩溃恢复，把生成的redo日志都放在了大小为512字节的块（block）中。

我们前边说过，为了解决磁盘速度过慢的问题而引入了Buffer Pool。同理，写入redo日志时也不能直接写到磁盘上，实际上在服务器启动时就向操作系统申请了一大片称之为redo log buffer的连续内存空间，翻译成中文就是redo日志缓冲区（内存），我们也可以简称为log buffer。这片内存空间被划分成若干个连续的redo log block，我们可以通过启动参数innodb_log_buffer_size来指定log buffer的大小，该启动参数的默认值为16MB。

```
mysql> show variables like 'innodb_log_buffer_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_log_buffer_size | 16777216 |
+-----+-----+
1 row in set (0.01 sec)
```

1.1.3.2.redo日志刷盘时机

可是这些日志总在内存里呆着也不是个办法，在一些情况下它们会被刷新到磁盘里，比如：

一、事务提交时，为了保证持久性，必须要把修改这些页面对应的redo日志刷新到磁盘。

不过这里有一个参数 innodb_flush_log_at_trx_commit 可以控制：

```
mysql> show variables like 'innodb_flush_log_at_trx_commit';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_flush_log_at_trx_commit | 1 |
+-----+-----+
1 row in set (0.00 sec)
```

该变量有3个可选的值：

0：当该系统变量值为0时，表示在事务提交时不立即向磁盘中同步redo日志，这个任务是交给后台线程做的。

这样很明显会加快请求处理速度，但是如果事务提交后服务器挂了，后台线程没有及时将redo日志刷新到磁盘，那么该事务对页面的修改会丢失。

1：当该系统变量值为1时，表示在事务提交时需要将redo日志同步到磁盘，可以保证事务的持久性。1也是innodb_flush_log_at_trx_commit的默认值。

2：当该系统变量值为2时，表示在事务提交时需要将redo日志写到操作系统的缓冲区中，但并不需要保证将日志真正的刷新到磁盘。

这种情况下如果数据库挂了，操作系统没挂的话，事务的持久性还是可以保证的，但是操作系统也挂了的话，那就不能保证持久性了。

二、InnoDB认为如果当前写入log buffer的redo日志量已经占满了log buffer总容量的大约一半左右，就需要把这些日志刷新到磁盘上。

三、后台有一个线程，大约每秒都会刷新一次log buffer中的redo日志到磁盘。

四、正常关闭服务器时等等。

1.1.4.崩溃后的恢复

1.1.4.1.恢复机制

在服务器不挂的情况下，redo日志简直就是个大累赘，不仅没用，反而让性能变得更差。但是万一数据库挂了，就可以在重启时根据redo日志中的记录就可以将页面恢复到系统崩溃前的状态。

MySQL可以根据redo日志中的各种信息，来确定恢复的起点和终点。然后将redo日志中的数据，以哈希表的形式，将一个页面下的放到哈希表的一个槽中。之后就可以遍历哈希表，因为对同一个页面进行修改的redo日志都放在了一个槽里，所以可以一次性将一个页面修复好（避免了很多读取页面的随机IO）。并且通过各种机制，避免无谓的页面修复，比如已经刷新的页面，进而提升崩溃恢复的速度。

1.1.4.2.崩溃后的恢复为什么不用binlog?

1、这两者使用方式不一样

binlog 会记录表所有更改操作，包括更新删除数据，更改表结构等等，主要用于人工恢复数据，而 redo log 对于我们是不可见的，它是 InnoDB 用于保证 crash-safe 能力的，也就是在事务提交后MySQL崩溃的话，可以保证事务的持久性，即事务提交后其更改是永久性的。

一句话概括：binlog 是用作人工恢复数据，redo log 是 MySQL 自己使用，用于保证在数据库崩溃时的事务持久性。

2、redo log 是 InnoDB 引擎特有的，binlog 是 MySQL 的 Server 层实现的,所有引擎都可以使用。

3、redo log是物理日志，记录的是“在某个数据页上做了什么修改”，恢复的速度更快；binlog是逻辑日志，记录的是这个语句的原始逻辑，比如“给ID=2这的c字段加1”；

4、redo log是“循环写”的日志文件，redo log 只会记录未刷盘的日志，已经刷入磁盘的数据都会从redo log 这个有限大小的日志文件里删除。binlog 是追加日志，保存的是全量的日志。

5、最重要的是，当数据库crash 后，想要恢复未刷盘但已经写入 redo log 和 binlog 的数据到内存时，binlog 是无法恢复的。虽然 binlog 拥有全量的日志，但没有一个标志让 innoDB 判断哪些数据已经入表(写入磁盘)，哪些数据还没有。

比如，binlog 记录了两条日志：

记录1：给 ID=2 这一行的 c 字段加1

记录2：给 ID=2 这一行的 c 字段加1

在记录1入表后，记录2未入表时，数据库crash。重启后，只通过 binlog 数据库无法判断这两条记录哪条已经写入磁盘，哪条没有写入磁盘，不管是两条都恢复至内存，还是都不恢复，对 ID=2 这行数据来说，都不对。

但 redo log 不一样，只要刷入磁盘的数据，都会从 redo log 中抹掉，数据库重启后，直接把 redo log 中的数据都恢复至内存就可以了。

1.2.undo日志

1.2.1.事务回滚的需求

我们说过事务需要保证原子性，也就是事务中的操作要么全部完成，要么什么也不做。但是偏偏有时候事务执行到一半会出现一些情况，比如：

情况一：事务执行过程中可能遇到各种错误，比如服务器本身的错误，操作系统错误，甚至是突然断电导致的错误。

情况二：程序员可以在事务执行过程中手动输入ROLLBACK语句结束当前的事务的执行。

这两种情况都会导致事务执行到一半就结束，但是事务执行过程中可能已经修改了很多东西，为了保证事务的原子性，我们需要把东西改回原先的样子，这个过程就称之为回滚（英文名：rollback），这样就可以造成这个事务看起来什么都没做，所以符合原子性要求。

每当我们要对一条记录做改动时（这里的改动可以指INSERT、DELETE、UPDATE），都需要把回滚时所需的东西都给记下来。比方说：

你插入一条记录时，至少要把这条记录的主键值记下来，之后回滚的时候只需要把这个主键值对应的记录删掉。

你删除了一条记录，至少要把这条记录中的内容都记下来，这样之后回滚时再把由这些内容组成的记录插入到表中。

你修改了一条记录，至少要把修改这条记录前的旧值都记录下来，这样之后回滚时再把这条记录更新为旧值。

这些为了回滚而记录的这些东西称之为撤销日志，英文名为undo log/undo日志。这里需要注意的一点是，由于查询操作（SELECT）并不会修改任何用户记录，所以在查询操作执行时，并不需要记录相应的undo日志。

当然，在真实的InnoDB中，undo日志其实并不像我们上边所说的那么简单，不同类型的操作产生的undo日志的格式也是不同的。

1.2.2.事务id

1.2.2.1.给事务分配id的时机

读写事务：

我们可以通过START TRANSACTION READ WRITE语句开启一个读写事务，或者使用BEGIN、START TRANSACTION语句开启的事务默认也算是读写事务。

在读写事务中可以对表执行增删改查操作。

如果某个事务执行过程中对某个表执行了增、删、改操作，那么InnoDB存储引擎就会给它分配一个独一无二的`事务id`，分配方式如下：

对于读写事务来说，只有在它第一次对某个表（包括用户创建的临时表）执行增、删、改操作时才会为这个事务分配一个`事务id`，否则的话也是不分配`事务id`的。

有的时候虽然我们开启了一个读写事务，但是在这个事务中全是查询语句，并没有执行增、删、改的语句，那也就意味着这个事务并不会被分配一个事务id。

上边描述的事务id分配策略是针对MySQL5.7来说的，前边的版本的分配方式可能不同。

1.2.2.2.事务id生成机制

这个事务id本质上就是一个数字，它的分配策略和我们前边提到的对隐藏列row_id（当用户没有为表创建主键和UNIQUE键时InnoDB自动创建的列）的分配策略大抵相同，具体策略如下：

服务器会在内存中维护一个全局变量，每当需要为某个事务分配一个事务id时，就会把该变量的值当作事务id分配给该事务，并且把该变量自增1。

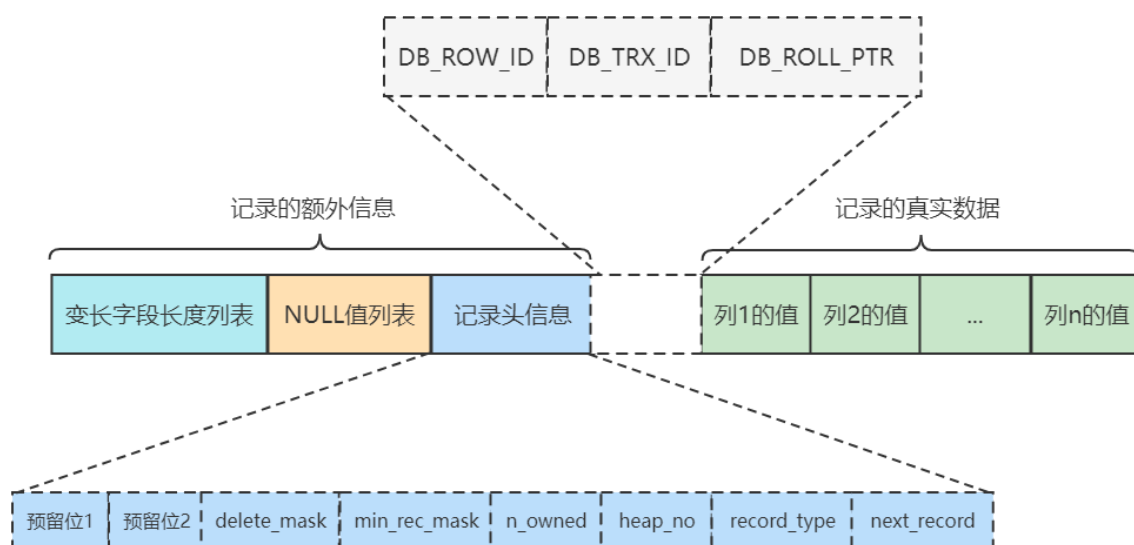
每当这个变量的值为256的倍数时，就会将该变量的值刷新到系统表空间的页号为5的页面中一个称之为Max Trx ID的属性处，这个属性占用8个字节的存储空间。

当系统下一次重新启动时，会将上边提到的Max Trx ID属性加载到内存中，将该值加上256之后赋值给我们前边提到的全局变量（因为在上次关机时该全局变量的值可能大于Max Trx ID属性值）。

这样就可以保证整个系统中分配的事务id值是一个递增的数字。先被分配id的事务得到的是较小的事务id，后被分配id的事务得到的是较大的事务id。

1.2.3.trx_id隐藏列

我们在学习InnoDB记录行格式的时候重点强调过：聚簇索引的记录除了会保存完整的用户数据以外，而且还会自动添加名为trx_id、roll_pointer的隐藏列，如果用户没有在表中定义主键以及UNIQUE键，还会自动添加一个名为row_id的隐藏列。



其中的trx_id列就是某个对这个聚簇索引记录做改动的语句所在的事务对应的事务id而已（此处的改动可以是INSERT、DELETE、UPDATE操作）。至于roll_pointer隐藏列我们后边分析。

1.2.4.undo日志的格式

为了实现事务的原子性，InnoDB存储引擎在实际进行增、删、改一条记录时，都需要先把对应的undo日志记下来。一般每对一条记录做一次改动，就对应着一条undo日志，但在某些更新记录的操作中，也可能会对对应着2条undo日志。

一个事务在执行过程中可能新增、删除、更新若干条记录，也就是说需要记录很多条对应的undo日志，这些undo日志会被从0开始编号，也就是说根据生成的顺序分别被称为第0号undo日志、第1号undo日志、...、第n号undo日志等，这个编号也被称之为undo no。

这些undo日志是被记录到类型为FIL_PAGE_UNDO_LOG的页面中。这些页面可以从系统表空间中分配，也可以从一种专门存放undo日志的表空间，也就是所谓的undo tablespace中分配。先来看看不同操作都会产生什么样子的undo日志。

1.2.4.1.INSERT操作对应的undo日志

当我们向表中插入一条记录时最终导致的结果就是这条记录被放到了一个数据页中。如果希望回滚这个插入操作，那么把这条记录删除就好了，也就是说在写对应的undo日志时，主要是把这条记录的主键信息记上。InnoDB的设计了一个类型为TRX_UNDO_INSERT_REC的undo日志。

我们知道，对于使用InnoDB存储引擎的表来说，它的聚簇索引记录中都包含两个必要的隐藏列（row_id并不是必要的，我们创建的表中有主键或者非NULL的UNIQUE键时都不会包含row_id列）：

trx_id：每次一个事务对某条聚簇索引记录进行改动时，都会把该事务的事务id赋值给trx_id隐藏列。

roll_pointer：每次对某条聚簇索引记录进行改动时，都会把旧的版本写入到undo日志中，然后这个隐藏列就相当于一个指针，可以通过它来找到该记录修改前的信息。

为了说明这个问题，我们创建一个演示表

```
CREATE TABLE teacher (  
  number INT,  
  name VARCHAR(100),  
  domain varchar(100),  
  PRIMARY KEY (number)  
) Engine=InnoDB CHARSET=utf8;
```

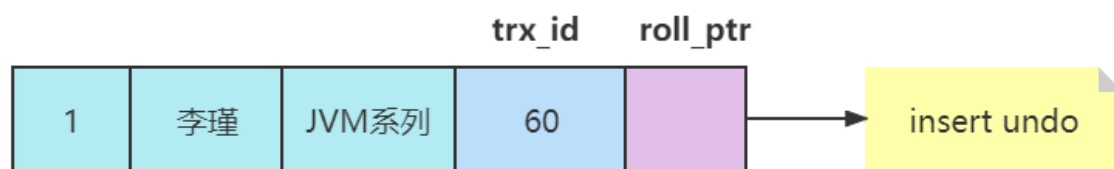
然后向这个表里插入一条数据：

```
INSERT INTO teacher VALUES(1, '李瑾', 'JVM系列');
```

现在表里的数据就是这样的：

number	name	domain
1	李瑾	JVM系列

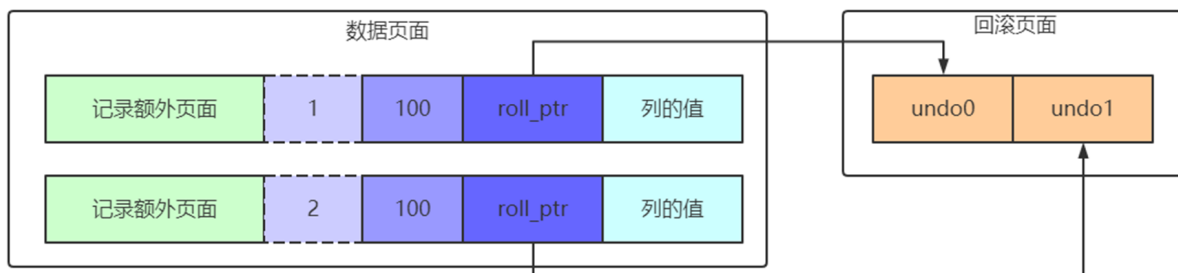
假设插入该记录的事务id为60，那么此刻该条记录的示意图如下所示：



如果记录中的主键只包含一个列，那么在类型为TRX_UNDO_INSERT_REC的undo日志中只需要把该列占用的存储空间大小和真实值记录下来，如果记录中的主键包含多个列，那么每个列占用的存储空间大小和对应的真实值都需要记录下来。

roll_pointer的作用

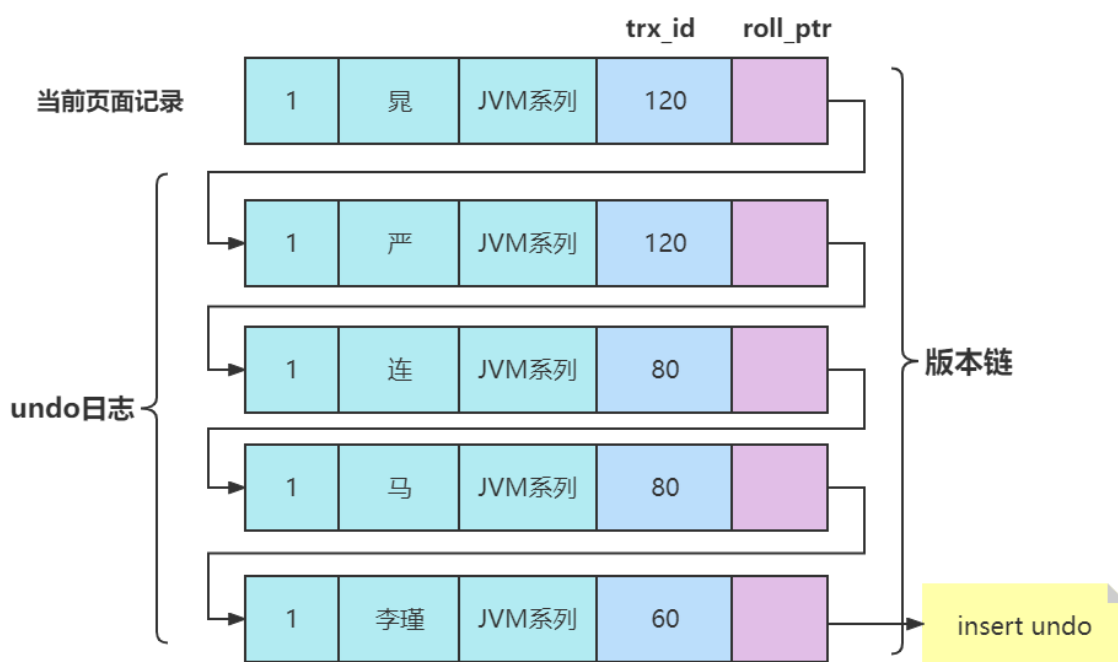
roll_pointer本质上就是一个指向记录对应的undo日志的一个指针。比方说我们向表里插入了2条记录，每条记录都有与其对应的一条undo日志。记录被存储到了类型为FIL_PAGE_INDEX的页面中（就是我们前边一直所说的数据页），undo日志被存放到了类型为FIL_PAGE_UNDO_LOG的页面中。roll_pointer本质就是一个指针，指向记录对应的undo日志。



1.2.4.2.DELETE操作对应的undo日志

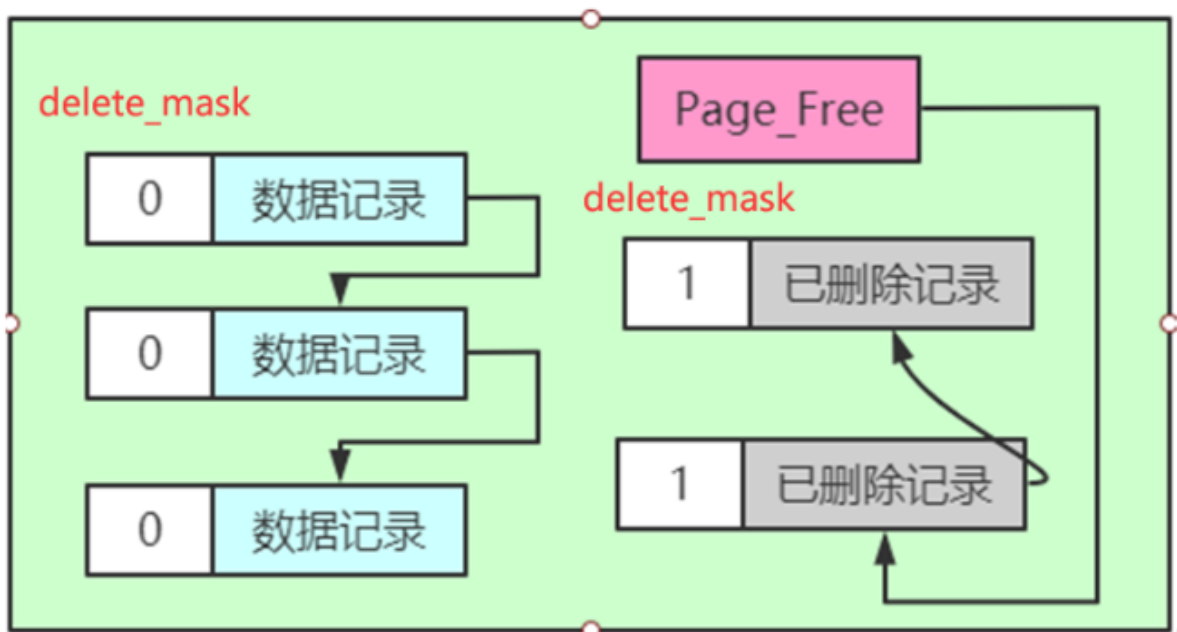
我们知道插入到页面中的记录会根据记录头信息中的next_record属性组成一个单向链表，我们把这个链表称之为正常记录链表；

往这张表中插入多条记录。每次对记录进行改动，都会记录一条undo日志，每条undo日志也都有一个roll_pointer属性（INSERT操作对应的undo日志没有该属性，因为该记录并没有更早的版本），可以将这些undo日志都连起来，串成一个链表，所以现在的情况就像下图一样：



被删除的记录其实也会根据记录头信息中的next_record属性组成一个链表，只不过这个链表中的记录占用的存储空间可以被重新利用，所以也称这个链表为垃圾链表。Page Header部分有一个称之为PAGE_FREE的属性，它指向由被删除记录组成的垃圾链表中的头节点。

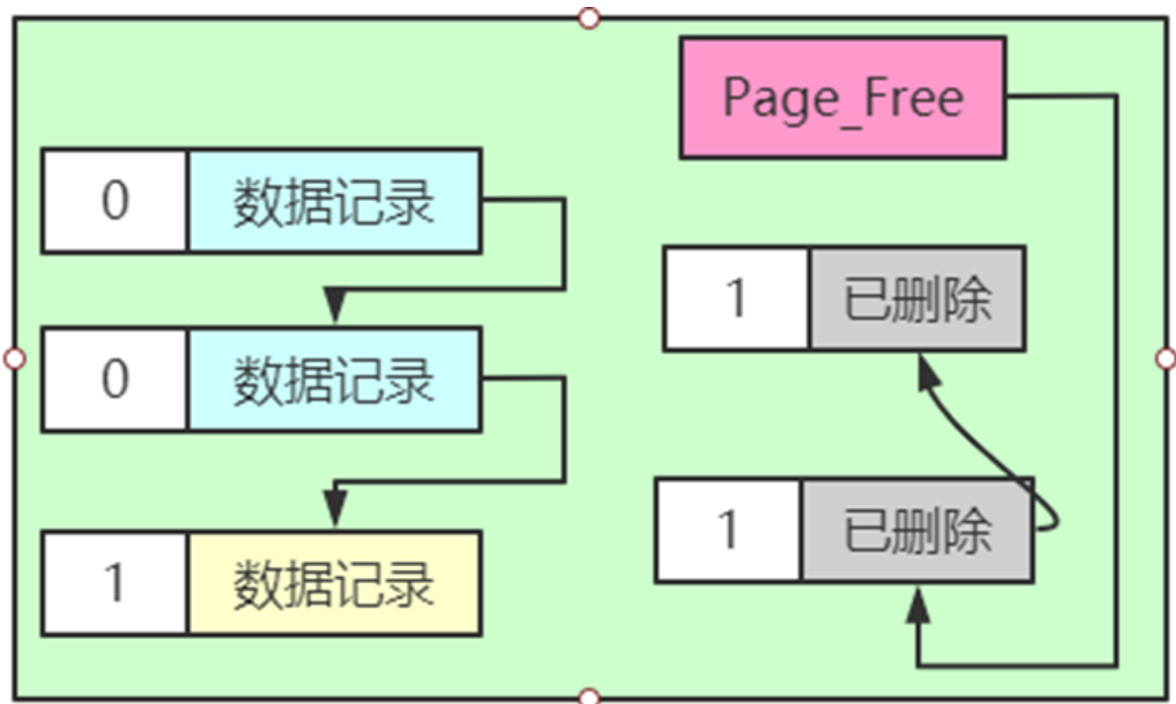
假设此刻某个页面中的记录分布情况是这样的



我们只把记录的delete_mask标志位展示了出来。从图中可以看出，正常记录链表中包含了3条正常记录，垃圾链表里包含了2条已删除记录。页面的Page Header部分的PAGE_FREE属性的值代表指向垃圾链表头节点的指针。

假设现在我们准备使用DELETE语句把正常记录链表中的最后一条记录给删除掉，其实这个删除的过程需要经历两个阶段：

阶段一：将记录的delete_mask标识位设置为1，这个阶段称之为delete mark。



可以看到，正常记录链表中的最后一条记录的delete_mask值被设置为1，但是并没有被加入到垃圾链表。也就是此时记录处于一个中间状态。在删除语句所在的事务提交之前，被删除的记录一直都处于这种所谓的中间状态。

为啥会有这种奇怪的中间状态呢？其实主要是为了实现MVCC中的事务隔离级别。

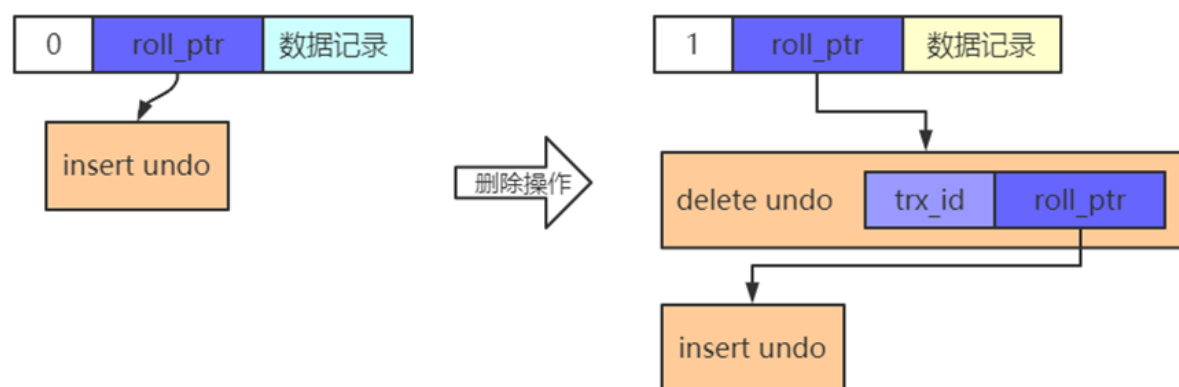
阶段二：当该删除语句所在的事务提交之后，会有专门的线程后来真正的把记录删除掉。所谓真正的删除就是把该记录从正常记录链表中移除，并且加入到垃圾链表中，然后还要调整一些页面的其他信息，比如页面中的用户记录数量PAGE_N_RECS、上次插入记录的位置PAGE_LAST_INSERT、垃圾链表头节点的指针PAGE_FREE、页面中可重用的字节数量PAGE_GARBAGE、还有页目录的一些信息等等。这个阶段称之为purge。

把阶段二执行完了，这条记录就算是真正的被删除掉了。这条已删除记录占用的存储空间也可以被重新利用了。

从上边的描述中我们也可以看出来，在删除语句所在的事务提交之前，只会经历阶段一，也就是delete mark阶段（提交之后我们就不用回滚了，所以只需考虑对删除操作的阶段一做的影响进行回滚）。InnoDB中就会产生一种称之为TRX_UNDO_DEL_MARK_REC类型的undo日志。

版本链

同时，在对一条记录进行delete mark操作前，需要把该记录的旧的trx_id和roll_pointer隐藏列的值都给记到对应的undo日志中来，就是我们图中显示的old trx_id和old roll_pointer属性。这样有一个好处，那就是可以通过undo日志的old roll_pointer找到记录在修改之前对应的undo日志。比方说在一个事务中，我们先插入了一条记录，然后又执行对该记录的删除操作，这个过程的示意图就是这样：



从图中可以看出，执行完delete mark操作后，它对应的undo日志和INSERT操作对应的undo日志就串成了一个链表。这个链表就称之为版本链。

1.2.4.3.UPDATE操作对应的undo日志

在执行UPDATE语句时，InnoDB对更新主键和不更新主键这两种情况有截然不同的处理方案。

不更新主键的情况

在不更新主键的情况下，又可以细分为被更新的列占用的存储空间不发生变化和发生变化的情况。

就地更新 (in-place update)

更新记录时，对于被更新的每个列来说，如果更新后的列和更新前的列占用的存储空间都一样大，那么就可以进行就地更新，也就是直接在原记录的基础上修改对应列的值。再次强调一边，是每个列在更新前后占用的存储空间一样大，有任何一个被更新的列更新前比更新后占用的存储空间大，或者更新前比更新后占用的存储空间小都不能进行就地更新。

先删除掉旧记录，再插入新记录

在不更新主键的情况下，如果有任何一个被更新的列更新前和更新后占用的存储空间大小不一致，那么就需要先把这条旧的记录从聚簇索引页面中删除掉，然后再根据更新后列的值创建一条新的记录插入到页面中。

请注意一下，我们这里所说的删除并不是delete mark操作，而是真正的删除掉，也就是把这条记录从正常记录链表中移除并加入到垃圾链表中，并且修改页面中相应的统计信息（比如PAGE_FREE、PAGE_GARBAGE等这些信息）。由用户线程同步执行真正的删除操作，真正删除之后紧接着就要根据各个列更新后的值创建的新记录插入。

这里如果新创建的记录占用的存储空间大小不超过旧记录占用的空间，那么可以直接重用被加入到垃圾链表中的旧记录所占用的存储空间，否则的话需要在页面中新申请一段空间以供新记录使用，如果本页面内已经没有可用的空间的话，那就需要进行页面分裂操作，然后再插入新记录。

针对UPDATE不更新主键的情况（包括上边所说的就地更新和先删除旧记录再插入新记录），InnoDB设计了一种类型为TRX_UNDO_UPD_EXIST_REC的undo日志。

更新主键的情况

在聚簇索引中，记录是按照主键值的大小连成了一个单向链表的，如果我们更新了某条记录的主键值，意味着这条记录在聚簇索引中的位置将会发生改变，比如你将记录的主键值从1更新为10000，如果还有非常多的记录的主键值分布在1 ~ 10000之间的话，那么这两条记录在聚簇索引中就有可能离得非常远，甚至中间隔了好多个页面。针对UPDATE语句中更新了记录主键值的这种情况，InnoDB在聚簇索引中分了两步处理：

将旧记录进行delete mark操作

也就是说在UPDATE语句所在的事务提交前，对旧记录只做一个delete mark操作，在事务提交后才由专门的线程做purge操作，把它加入到垃圾链表中。这里一定要和我们上边所说的在不更新记录主键值时，先真正删除旧记录，再插入新记录的方式区分开！

之所以只对旧记录做delete mark操作，是因为别的事务同时也可能访问这条记录，如果把它真正的删除加入到垃圾链表后，别的事务就访问不到了。这个功能就是所谓的MVCC。

创建一条新记录

根据更新后各列的值创建一条新记录，并将其插入到聚簇索引中（需重新定位插入的位置）。

由于更新后的记录主键值发生了改变，所以需要重新从聚簇索引中定位这条记录所在的位置，然后把它插进去。

针对UPDATE语句更新记录主键值的这种情况：

在对该记录进行delete mark操作前，会记录一条类型为TRX_UNDO_DEL_MARK_REC的undo日志；

之后插入新记录时，会记录一条类型为TRX_UNDO_INSERT_REC的undo日志，也就是说每对一条记录的主键值做改动时，会记录2条undo日志。